Relational Database vs NoSQL Database

A comparison of data models and use cases using the library system

Prepared by: Vigneshvar K K

Project: Library Management System. **Output convention:** result tables below are reproducible expected outputs from the fixed rows in the shared SQL script. They are not screenshots of MySQL execution.

1. Purpose

A relational database stores related facts in tables and connects them with keys. A NoSQL database is a broad group of non-relational models, including document, key-value, wide-column, and graph databases. The choice depends on the shape of the data, access patterns, consistency needs, and operating scale.

2. Comparison for the library project

Dimension	Relational design	Document / NoSQL design
Data model	Separate tables for books, authors, copies, members, loans, and reservations; keys connect the rows.	A book document could embed its author list and descriptive fields; copy or circulation records may be embedded or stored separately.
Consistency	Foreign keys, unique keys, checks, and transactions can protect related circulation changes.	Rules depend on the product and data model. Embedding can make a grouped update atomic; references can require multi-document coordination.
Scaling	Often begins with vertical scaling and can scale out through replicas, partitioning, or sharding when designed for it.	Many products provide horizontal partitioning options; data distribution and query patterns still need careful design.
Query style	SQL joins and aggregate queries fit reports across members, loans, copies, and books.	Document queries can retrieve a grouped document efficiently; cross-collection reporting may need additional aggregation or duplicated fields.
Schema change	Tables have declared columns and constraints. Planned migrations help keep data consistent.	Documents can vary by record, which suits evolving or optional attributes; the application still needs validation rules.
Good fit	Library circulation, banking transfers, student enrollment, or inventory where links and multi-row rules matter.	Product catalogs with varying attributes, content records, device events, or user profiles read as a whole document.

3. Library example

The relational model stores a book title once, connects its authors through `book_authors`, and stores each copy separately. A loan references a specific physical copy and a member. This model makes a report such as 'open loans by member and title' a predictable join and lets a foreign key reject a loan for a nonexistent copy.

A document model could return a catalog entry and its descriptive author list in one read. It may be a useful fit for a read-heavy public catalog with flexible metadata. Circulation still needs rules to prevent two concurrent checkouts of the same copy and to keep member, copy, loan, and reservation data aligned. A document database can support consistency features too; the design and transaction boundaries matter.

Illustrative document shape

```
{
  "bookId": 1,
  "isbn": "9780133970777",
  "title": "Relational Database Basics",
  "publisher": {"name": "Meridian Academic", "city": "Mumbai"},
  "authors": ["Ada Sen", "Ravi Menon"],
  "category": "Database"
}
```

This is only an illustration of a document representation. The submitted implementation remains the normalized MySQL schema used in Assignments 3 and 5-9.

4. Choosing a technology in practice

Question	Why it matters here
Are the links essential?	A checkout must connect an existing member to a particular copy and keep its loan history.
Do reports cross entities?	The lecturer asks for multi-table reports, summaries, and related records, which are direct SQL join patterns.
How variable are catalog fields?	Optional catalog metadata could fit flexible documents, while core member and circulation fields still need validation.
What is the expected scale?	A BCA-sized library can begin with one relational database; scaling choices can be revisited if use and load grow.

The technology decision follows the workload. A flexible catalog view and a circulation system can use different storage models in a larger product, while this assignment keeps one MySQL design so the schema, constraints, and query reports stay consistent.

5. Decision

For this BCA project, MySQL 8.0 is selected because circulation links among members, copies, loans, and reservations are central, and the assignment requires relational DDL, constraints, joins, and aggregate reports. A document database could complement a high-volume catalog search service, but it would not remove the need to define ownership, update, and consistency rules.

6. References

MySQL describes InnoDB as transaction-capable and documents its isolation levels. MongoDB's data-modeling guidance explains that embedded data can reduce read operations and make grouped updates atomic, while repeated fields may result from denormalized models.

MySQL 8.0 Reference Manual, InnoDB and the ACID Model: https://dev.mysql.com/doc/refman/8.0/en/mysql-acid.html
MySQL 8.0 Reference Manual, Transaction Isolation Levels: https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html
MongoDB Manual, Embedded Data: https://www.mongodb.com/docs/manual/data-modeling/embedding/

7. Screenshot guidance

This comparison is conceptual, so a query screenshot is not required. If the GitHub submission includes execution evidence, capture `SELECT VERSION();` in MySQL Workbench to document the server used for the later SQL assignments.